



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Reti Sociali

# Majority Cascade Dynamics in Online Networks

Progetto di Reti Sociali

---

**Studente**

Giacomo Favale

Matricola: 0522501990

**Docente/i**

Prof.ssa Luisa Gargano

Prof.ssa Anna Adele Rescigno

Anno Accademico 2025/2026

## Abstract

Questa relazione presenta un progetto sulla diffusione dell'influenza in reti complesse, utilizzando il modello Majority Cascade con vincoli di costo. L'obiettivo è scegliere un *seed set*  $S \subseteq V$  che rispetti un budget massimo  $c(S) \leq k$  e che riesca ad attivare il maggior numero possibile di nodi nella rete. Nel modello considerato, un nodo  $v$  si attiva quando almeno  $\lceil d(v)/2 \rceil$  dei suoi vicini sono già attivi. Il processo avviene in modo sincrono e termina quando non ci sono più nuovi nodi da attivare.

Gli esperimenti sono stati svolti sulla rete peer-to-peer **p2p-Gnutella04**, composta da 10.876 nodi e 39.994 archi. Anche se il dataset originale è diretto, nel progetto la rete è stata considerata come non orientata, perché il Majority Cascade utilizzato si basa su vicinato, grado e soglia di maggioranza, senza distinguere tra archi entranti e uscenti. Sono state analizzate tre funzioni costo: *Random Cost*, *Degree Cost* e *Sublinear Degree Cost*. Per la scelta del seed set sono stati confrontati cinque algoritmi: *Greedy f1*, *Greedy f2*, *Greedy f3*, *WTSS* e *Threshold-Deficit Greedy*, un'euristica proposta nel progetto e basata sul deficit di attivazione dei nodi.

Dai risultati emerge che la funzione costo scelta ha un impatto importante sulla diffusione finale. In particolare, la *Sublinear Degree Cost* risulta interessante perché considera la struttura della rete, ma penalizza i nodi ad alto grado in modo meno forte rispetto alla *Degree Cost*. L'algoritmo *Threshold-Deficit Greedy* ottiene buoni risultati soprattutto con budget bassi e medi, spesso superando gli altri algoritmi in termini di nodi attivati. Infine, sono stati svolti anche stress test con rimozione casuale di archi e nodi, per valutare quanto i seed set trovati rimangono efficaci dopo alcune modifiche alla struttura della rete.

# Indice

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                          | <b>1</b>  |
| <b>2</b> | <b>Metodologia</b>                           | <b>3</b>  |
| 2.1      | La rete utilizzata per gli esperimenti       | 3         |
| 2.2      | Funzioni di costo                            | 5         |
| 2.3      | Algoritmi utilizzati                         | 7         |
| 2.3.1    | Greedy Seed Set Selection                    | 7         |
| 2.3.2    | WTSS                                         | 8         |
| 2.3.3    | Threshold-Deficit Greedy                     | 8         |
| 2.4      | Majority Cascade in Networks                 | 10        |
| 2.5      | Valutazione degli algoritmi                  | 11        |
| <b>3</b> | <b>Risultati</b>                             | <b>13</b> |
| 3.1      | Confronto tra funzioni di costo              | 13        |
| 3.1.1    | Random Cost Function                         | 13        |
| 3.1.2    | Half-Node-Degree Cost Function               | 15        |
| 3.1.3    | Sublinear Degree Cost Function               | 17        |
| 3.2      | Confronto complessivo dei migliori risultati | 18        |
| 3.3      | Analisi dei tempi di esecuzione              | 19        |
| 3.4      | Valutazione della robustezza                 | 20        |
| 3.4.1    | Rimozione casuale di archi                   | 21        |
| 3.4.2    | Rimozione casuale di nodi                    | 23        |
| 3.5      | Discussione finale                           | 25        |
| <b>4</b> | <b>Conclusioni</b>                           | <b>27</b> |

# 1 Introduzione

La diffusione dell'influenza all'interno di una rete è un tema importante nello studio delle reti sociali e, più in generale, delle reti complesse. In molti contesti reali, come social network, reti peer-to-peer o reti di comunicazione, un'informazione o un comportamento può partire da un piccolo insieme iniziale di nodi e poi propagarsi progressivamente al resto della rete. Per descrivere questo tipo di fenomeno si possono usare modelli di diffusione a cascata, nei quali lo stato di un nodo dipende dallo stato dei suoi vicini.

In questa relazione viene considerato il modello **Majority Cascade**. In questo modello, dato un grafo  $G = (V, E)$  e un insieme iniziale di nodi attivi  $S \subseteq V$ , detto *seed set*, un nodo non ancora attivo si attiva quando almeno la metà dei suoi vicini è già attiva. Indicando con  $N(v)$  il vicinato di un nodo  $v$  e con  $d(v)$  il suo grado, la soglia di attivazione è quindi:

$$\tau(v) = \left\lceil \frac{d(v)}{2} \right\rceil.$$

Il processo parte da  $Inf[S, 0] = S$  e procede a round successivi. A ogni round vengono aggiunti all'insieme dei nodi attivi tutti i nodi che soddisfano la soglia di maggioranza. Una volta attivato, un nodo rimane attivo fino alla fine del processo. La cascata termina quando non vengono attivati nuovi nodi, cioè quando si raggiunge una configurazione stabile. L'insieme finale dei nodi attivati viene indicato con  $Inf[G, S]$ .

Nel progetto viene considerata anche la variante con costi, chiamata **Cost Majority Cascade**. In questo caso a ogni nodo  $u \in V$  è associato un costo  $c(u)$  e la scelta del seed set deve rispettare un budget massimo  $k$ . Il costo totale di un seed set è dato da:

$$c(S) = \sum_{u \in S} c(u).$$

L'obiettivo è quindi scegliere un insieme di seed  $S$  tale che:

$$c(S) \leq k$$

e che massimizzi il numero finale di nodi attivati, cioè  $|Inf[G, S]|$ . Questo problema

non è semplice da risolvere in modo esatto, soprattutto su reti di grandi dimensioni. Per questo motivo, come indicato anche dalle slide del progetto, si ricorre ad algoritmi euristici, che cercano di ottenere buone soluzioni rispettando il vincolo di budget.

La sperimentazione è stata svolta sulla rete **p2p-Gnutella04**, appartenente alla collezione SNAP<sup>1</sup>. Si tratta di una rete peer-to-peer Gnutella raccolta il 4 agosto 2002, composta da 10.876 nodi e 39.994 archi. Il dataset originale è diretto, ma in questo lavoro è stato trattato come grafo non orientato, perché il modello Majority Cascade utilizzato si basa su vicinato, grado e soglia di maggioranza, senza distinguere tra archi entranti e archi uscenti. Questa scelta permette quindi di applicare in modo coerente la definizione del processo di cascata. Nel lavoro sono state considerate tre diverse funzioni costo. La prima è una funzione randomica, in cui il costo dei nodi viene scelto casualmente in un intervallo fissato. La seconda è una funzione basata sul grado, in cui il costo di un nodo dipende da  $\lceil d(u)/2 \rceil$ . La terza, proposta nel progetto, è una funzione sublineare basata sul grado, pensata per tenere conto della struttura della rete senza penalizzare eccessivamente i nodi ad alto grado.

Per la selezione del seed set sono stati confrontati cinque algoritmi. Sono state utilizzate tre varianti greedy, indicate come *Greedy f1*, *Greedy f2* e *Greedy f3*, l'algoritmo *WTSS*, adattato al caso con vincolo di budget, e un'euristica proposta nel progetto, chiamata *Threshold-Deficit Greedy*. Quest'ultima sceglie i nodi cercando di favorire l'attivazione dei vicini che sono più vicini al raggiungimento della propria soglia majority.

Gli esperimenti sono stati organizzati confrontando gli algoritmi al variare della funzione costo e del budget disponibile. Per ogni configurazione è stato calcolato il seed set, è stata eseguita la cascata e sono stati misurati il numero finale di nodi attivati, il rapporto di diffusione rispetto al numero totale di nodi e il tempo di esecuzione. Inoltre, per valutare la robustezza dei seed set ottenuti, sono stati svolti stress test in cui la rete viene modificata tramite rimozione casuale di archi e nodi. In questo modo è possibile osservare quanto l'influenza prodotta dal seed set rimane stabile anche dopo cambiamenti nella struttura della rete.

La relazione è organizzata come segue. Nel Capitolo 2 viene descritta la metodologia adottata, includendo la rete utilizzata, le funzioni costo, gli algoritmi e le metriche di valutazione. Nel Capitolo 3 vengono presentati e discussi i risultati sperimentali. Infine, nel Capitolo 4 vengono riportate le conclusioni.

---

<sup>1</sup><https://snap.stanford.edu/data/p2p-Gnutella04.html>

## 2 Metodologia

In questo capitolo viene descritta la metodologia seguita per svolgere gli esperimenti. In particolare, vengono presentati la rete utilizzata, le funzioni di costo associate ai nodi, gli algoritmi scelti per la selezione del *seed set*, il modello di diffusione Majority Cascade e le metriche usate per valutare i risultati.

L'obiettivo degli esperimenti è confrontare diverse strategie di selezione dei nodi inizialmente attivi, verificando quanto riescano a diffondere l'influenza nella rete senza superare un budget prefissato. Inoltre, viene valutata la robustezza dei seed set ottenuti, modificando la rete tramite rimozione casuale di archi e nodi.

### 2.1 La rete utilizzata per gli esperimenti

Per la valutazione sperimentale è stata scelta la rete **p2p-Gnutella04**, disponibile nella collezione SNAP<sup>2</sup>. Si tratta di una rete peer-to-peer Gnutella, in cui i nodi rappresentano gli host della rete e gli archi rappresentano le connessioni tra essi.

Il dataset originale è diretto. Tuttavia, nel progetto il grafo è stato trattato come non orientato, perché il modello Majority Cascade utilizzato si basa sul vicinato  $N(v)$ , sul grado  $d(v)$  e sulla soglia di maggioranza dei nodi, senza distinguere tra archi entranti e archi uscenti. Inoltre, eventuali *self-loops* sono stati rimossi, in modo da non introdurre contributi non significativi nel processo di attivazione.

Le principali proprietà strutturali della rete sono riportate nella Tabella 1.

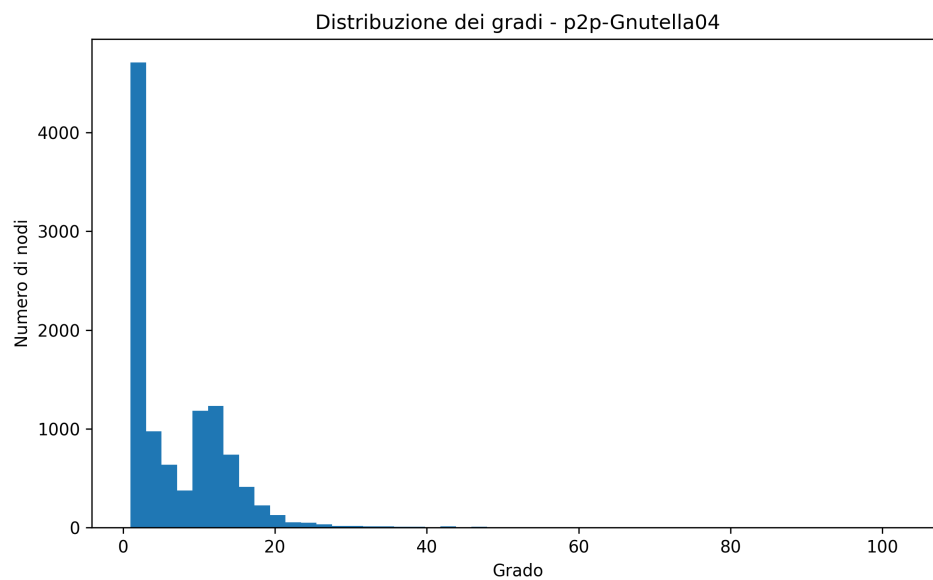
---

<sup>2</sup><https://snap.stanford.edu/data/p2p-Gnutella04.html>

**Tabella 1:** Proprietà strutturali della rete p2p-Gnutella04.

| Proprietà                           | Valore   |
|-------------------------------------|----------|
| Numero di nodi                      | 10.876   |
| Numero di archi                     | 39.994   |
| Grado minimo                        | 1        |
| Grado massimo                       | 103      |
| Grado medio                         | 7,35     |
| Densità                             | 0,000676 |
| Numero di componenti connesse       | 1        |
| Dimensione della componente gigante | 10.876   |
| Coefficiente medio di clustering    | 0,0062   |
| Numero di triangoli                 | 934      |
| Diametro della componente gigante   | 10       |

La rete risulta connessa e molto sparsa, poiché la densità è bassa rispetto al numero di nodi presenti. La distribuzione dei gradi, mostrata in Figura 1, evidenzia che la maggior parte dei nodi ha grado basso, mentre pochi nodi hanno un grado più elevato.

**Figura 1:** Distribuzione dei gradi della rete p2p-Gnutella04.

Questa caratteristica è importante per il Majority Cascade, perché la soglia di attivazione di un nodo è pari a  $\lceil d(v)/2 \rceil$ . Di conseguenza, i nodi con grado basso

possono essere attivati con pochi vicini attivi, mentre i nodi con grado alto sono più difficili da attivare.

## 2.2 Funzioni di costo

Nel problema Cost Majority Cascade, a ogni nodo  $u \in V$  è associato un costo di selezione  $c(u)$ . Il costo totale di un seed set  $S$  è dato da:

$$c(S) = \sum_{u \in S} c(u).$$

L'obiettivo è quindi scegliere un insieme di nodi inizialmente attivi che rispetti il vincolo  $c(S) \leq k$ , dove  $k$  rappresenta il budget disponibile.

Nel progetto sono state considerate tre funzioni di costo, in modo da valutare il comportamento degli algoritmi in scenari diversi.

La prima è la **Random Cost**, che assegna a ogni nodo un costo casuale intero compreso tra 1 e 10:

$$c(u) \in \{1, \dots, 10\}.$$

Questa funzione non dipende dalla struttura della rete. Di conseguenza, un nodo con alto grado può avere costo basso, così come un nodo poco connesso può avere costo alto.

La seconda è la **Degree Cost**, definita come:

$$c(u) = \left\lceil \frac{d(u)}{2} \right\rceil.$$

In questo caso il costo dipende direttamente dal grado del nodo. I nodi più connessi, che possono essere utili per la diffusione, diventano quindi più costosi da selezionare.

La terza è la **Sublinear Degree Cost**, definita come:

$$c_{sub}(u) = 1 + \lceil \log_2(1 + d(u)) \rceil.$$

Questa funzione è stata proposta nel progetto come compromesso tra le due precedenti. Rispetto alla Random Cost, mantiene un legame con la struttura della rete; rispetto alla Degree Cost, penalizza i nodi ad alto grado in modo meno forte.

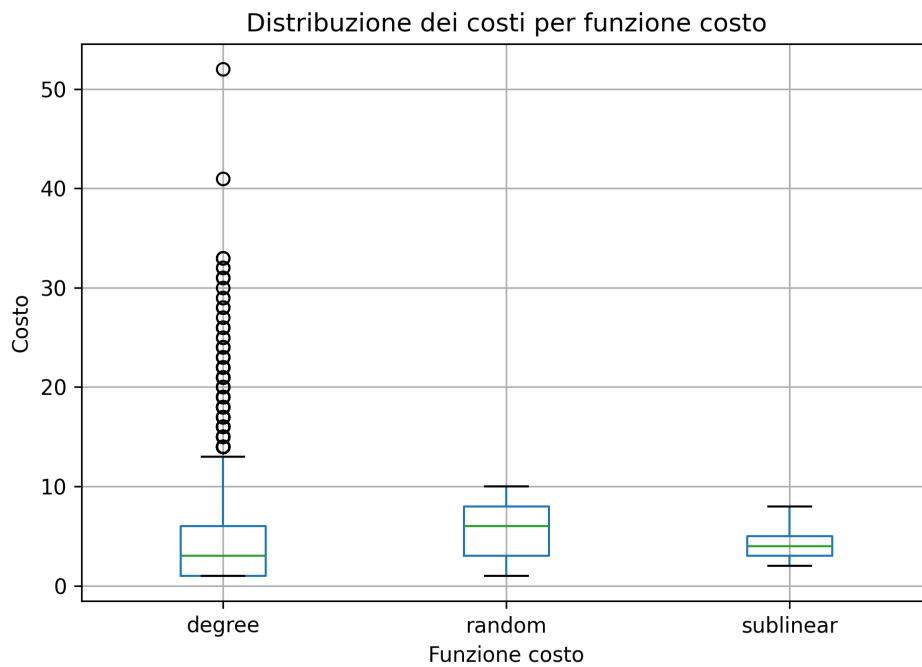
Le principali statistiche delle funzioni di costo sono riportate nella Tabella 2.



**Tabella 2:** Statistiche delle funzioni di costo considerate.

| Funzione costo        | Min | Max | Media | Mediana | Costo totale |
|-----------------------|-----|-----|-------|---------|--------------|
| Random Cost           | 1   | 10  | 5,52  | 6       | 60.070       |
| Degree Cost           | 1   | 52  | 3,97  | 3       | 43.166       |
| Sublinear Degree Cost | 2   | 8   | 3,86  | 4       | 42.010       |

La Figura 2 mostra la distribuzione dei costi generati dalle tre funzioni.

**Figura 2:** Distribuzione dei costi per le tre funzioni considerate.

Per ciascuna funzione costo sono stati definiti sei valori di budget, corrispondenti allo 0,5%, 1%, 2%, 5%, 10% e 20% del costo totale della rete. I valori usati negli esperimenti sono riportati nella Tabella 3.

**Tabella 3:** Valori di budget utilizzati per ciascuna funzione costo.

| Budget (%) | Random | Degree | Sublinear |
|------------|--------|--------|-----------|
| 0,5%       | 300    | 215    | 210       |
| 1%         | 600    | 431    | 420       |
| 2%         | 1.201  | 863    | 840       |
| 5%         | 3.003  | 2.158  | 2.100     |
| 10%        | 6.007  | 4.316  | 4.201     |
| 20%        | 12.014 | 8.633  | 8.402     |

## 2.3 Algoritmi utilizzati

Per la selezione del seed set sono stati implementati e confrontati cinque algoritmi: tre varianti greedy, WTSS e una euristica proposta nel progetto, chiamata Threshold-Deficit Greedy. Tutti gli algoritmi ricevono in input il grafo  $G$ , una funzione costo  $c$  e un budget  $k$ , e restituiscono un seed set  $S$  con costo non superiore al budget.

### 2.3.1 Greedy Seed Set Selection

Il primo approccio considerato è il **Cost-Seeds-Greedy**. L'algoritmo costruisce il seed set in modo incrementale. A ogni passo seleziona il nodo che massimizza il rapporto tra il guadagno marginale di una funzione di potenziale e il costo del nodo:

$$u^* = \arg \max_{u \in V \setminus S} \frac{\Delta_u f_i(S)}{c(u)}.$$

In questa formula,  $\Delta_u f_i(S)$  indica l'incremento della funzione di potenziale ottenuto aggiungendo il nodo  $u$  al seed set corrente.

Sono state considerate tre varianti dell'algoritmo, indicate come **Greedy f1**, **Greedy f2** e **Greedy f3**. Le tre versioni seguono la stessa logica greedy, ma usano funzioni di potenziale diverse per valutare il contributo dei nodi candidati.

Durante la costruzione del seed set, l'algoritmo controlla che il costo complessivo non superi il budget disponibile. Se l'aggiunta di un nodo porterebbe a superare il vincolo  $c(S) \leq k$ , viene mantenuto il seed set valido ottenuto prima di tale aggiunta.

### 2.3.2 WTSS

Il secondo algoritmo considerato è **WTSS** (*Working Target Set Selection*). WTSS è una procedura euristica che seleziona un insieme iniziale di nodi in grado di favorire una cascata di attivazioni.

L'algoritmo lavora rimuovendo progressivamente nodi da un grafo residuo e aggiornando, a ogni passo, i gradi e le soglie residue. Durante l'esecuzione possono verificarsi tre situazioni principali: un nodo può essere già attivabile dai vicini rimossi, può non avere più vicini sufficienti per raggiungere la soglia, oppure può essere rimosso in base a un criterio che considera costo, soglia e grado residuo.

Nel progetto, WTSS è stato adattato al caso con vincolo di budget. In particolare, quando l'aggiunta di un nuovo nodo al seed set farebbe superare il budget, viene mantenuto l'ultimo seed set valido, cioè quello con costo complessivo non superiore a  $k$ .

### 2.3.3 Threshold-Deficit Greedy

Il terzo approccio considerato è una euristica progettuale denominata **Threshold-Deficit Greedy**. L'idea di base è sfruttare direttamente la logica del Majority Cascade.

Nel Majority Cascade, ogni nodo  $v$  ha una soglia di attivazione:

$$\tau(v) = \left\lceil \frac{d(v)}{2} \right\rceil.$$

Durante la costruzione del seed set, per ogni nodo non ancora attivo viene considerato un deficit:

$$def(v) = \tau(v) - a(v),$$

dove  $a(v)$  indica il numero di vicini già attivi di  $v$ .

L'algoritmo assegna a ogni nodo candidato uno score, con l'obiettivo di dare maggiore importanza ai nodi che possono avvicinare i propri vicini alla soglia di attivazione. Il nodo scelto è quello con il miglior rapporto tra score e costo:

$$\frac{score(u)}{c(u)}.$$

**Algorithm 1:** Threshold-Deficit Greedy**Input:** Grafo non orientato  $G = (V, E)$ , funzione costo  $c : V \rightarrow \mathbb{N}$ , budget  $k$ **Output:** Seed set  $S \subseteq V$  tale che  $c(S) \leq k$ 


---

```

1  $S \leftarrow \emptyset$ ;
2  $A \leftarrow \emptyset$ ;
3  $B \leftarrow 0$ ;
4 foreach  $v \in V$  do
5    $\tau(v) \leftarrow \lceil \frac{d(v)}{2} \rceil$ ;
6    $a(v) \leftarrow 0$ ;
7    $def(v) \leftarrow \tau(v)$ ;
8 while  $B < k$  and  $A \neq V$  do
9    $bestNode \leftarrow \perp$ ;
10   $bestValue \leftarrow -\infty$ ;
11  foreach  $u \in V \setminus A$  do
12    if  $B + c(u) \leq k$  then
13       $score(u) \leftarrow 1$ ;
14      foreach  $v \in N(u) \setminus A$  do
15        if  $def(v) > 0$  then
16           $score(u) \leftarrow score(u) + \frac{1}{def(v)}$ ;
17       $value(u) \leftarrow \frac{score(u)}{c(u)}$ ;
18      if  $value(u) > bestValue$  then
19         $bestValue \leftarrow value(u)$ ;
20         $bestNode \leftarrow u$ ;
21  if  $bestNode = \perp$  then
22    break;
23   $S \leftarrow S \cup \{bestNode\}$ ;
24   $B \leftarrow B + c(bestNode)$ ;
25   $NewActive \leftarrow \{bestNode\}$ ;
26  while  $NewActive \neq \emptyset$  do
27     $A \leftarrow A \cup NewActive$ ;
28     $NextActive \leftarrow \emptyset$ ;
29    foreach  $u \in NewActive$  do
30      foreach  $v \in N(u) \setminus A$  do
31         $a(v) \leftarrow a(v) + 1$ ;
32         $def(v) \leftarrow \tau(v) - a(v)$ ;
33        if  $def(v) \leq 0$  then
34           $NextActive \leftarrow NextActive \cup \{v\}$ ;
35     $NewActive \leftarrow NextActive \setminus A$ ;
36 return  $S$ ;

```

---

Lo score utilizzato dall'algoritmo è:

$$score(u) = 1 + \sum_{\substack{v \in N(u) \\ v \notin A \\ def(v) > 0}} \frac{1}{def(v)}.$$

Il termine 1 rappresenta l'attivazione diretta del nodo scelto come seed, mentre il termine sommatorio misura il contributo dato ai vicini non ancora attivi. In particolare, i vicini con deficit più basso pesano di più, perché sono più vicini alla propria soglia di attivazione.

Threshold-Deficit Greedy non viene presentato come un nuovo algoritmo teorico, ma come una euristica progettuale coerente con il Majority Cascade. Il suo scopo è confrontare una strategia basata esplicitamente sulle soglie residue con le varianti greedy standard e con WTSS.

## 2.4 Majority Cascade in Networks

Dopo aver selezionato il seed set, viene simulato il processo di diffusione tramite Majority Cascade. Dato un grafo non orientato  $G = (V, E)$  e un seed set iniziale  $S \subseteq V$ , all'inizio risultano attivi solo i nodi appartenenti a  $S$ :

$$Inf[S, 0] = S.$$

A ogni round successivo, un nodo inattivo  $v$  si attiva se almeno la metà dei suoi vicini è già attiva:

$$|N(v) \cap Inf[S, r-1]| \geq \left\lceil \frac{d(v)}{2} \right\rceil.$$

L'insieme dei nodi attivi al round  $r$  è quindi:

$$Inf[S, r] = Inf[S, r-1] \cup \left\{ v \in V \setminus Inf[S, r-1] : |N(v) \cap Inf[S, r-1]| \geq \left\lceil \frac{d(v)}{2} \right\rceil \right\}.$$

Il processo è sincrono: le attivazioni di un round vengono calcolate usando solo lo stato del round precedente. La simulazione termina quando non vengono attivati nuovi nodi. L'insieme finale dei nodi attivati viene indicato con:

$$Inf[G, S].$$

Questa simulazione serve a valutare la qualità effettiva del seed set prodotto dagli

algoritmi. Infatti, un seed set non viene valutato solo per la sua dimensione o per il suo costo, ma soprattutto per il numero di nodi che riesce ad attivare.

## 2.5 Valutazione degli algoritmi

La valutazione sperimentale è stata divisa in due parti. La prima riguarda il confronto degli algoritmi sul grafo originale  $G$ , mentre la seconda riguarda la robustezza dei seed set quando la rete viene modificata.

Nel confronto principale, ogni algoritmo è stato eseguito per ciascuna combinazione di funzione costo e budget. Sono quindi state considerate tre funzioni costo, cinque algoritmi e sei valori di budget, per un totale di:

$$3 \times 5 \times 6 = 90$$

esecuzioni sperimentali.

Per ogni esecuzione sono state registrate le seguenti informazioni:

- funzione di costo utilizzata;
- algoritmo applicato;
- percentuale di budget;
- valore assoluto del budget  $k$ ;
- dimensione del seed set  $|S|$ ;
- costo complessivo del seed set  $c(S)$ ;
- numero finale di nodi attivati  $|Inf[G, S]|$ ;
- *diffusion ratio*;
- tempo di esecuzione.

Il *diffusion ratio* è definito come:

$$\frac{|Inf[G, S]|}{|V|}.$$

Questa metrica indica la frazione di nodi della rete attivati dalla cascata e permette di confrontare più facilmente algoritmi e configurazioni diverse.

Per ogni esecuzione è stato anche controllato il rispetto del vincolo:

$$c(S) \leq k.$$

Questo controllo è necessario perché un seed set con costo superiore al budget non

sarebbe una soluzione valida, anche se producesse una maggiore diffusione.

La seconda parte della valutazione riguarda gli stress test. A partire da alcune configurazioni rappresentative, il seed set selezionato sul grafo originale viene testato su un grafo modificato  $G'$ . Le modifiche considerate sono:

- rimozione casuale di archi;
- rimozione casuale di nodi.

Nel caso della rimozione di archi, il seed set  $S$  rimane invariato e viene calcolato  $Inf[G', S]$ . Nel caso della rimozione di nodi, invece, alcuni seed potrebbero essere stati eliminati dal grafo; per questo motivo viene usato il seed set aggiornato:

$$S' = S \cap V(G').$$

In questo caso viene calcolato  $Inf[G', S']$ .

Gli stress test sono stati eseguiti rimuovendo casualmente l'1%, il 5%, il 10% e il 20% degli archi o dei nodi. Il confronto tra la diffusione ottenuta sul grafo originale e quella ottenuta sul grafo modificato permette di valutare quanto i seed set individuati rimangano efficaci anche in presenza di cambiamenti nella struttura della rete.

## 3 Risultati

In questa sezione vengono presentati i risultati ottenuti dagli esperimenti descritti nel capitolo precedente. L'obiettivo è confrontare gli algoritmi di selezione del *seed set* in base alla loro capacità di attivare il maggior numero possibile di nodi, rispettando sempre il vincolo di budget.

Per ogni combinazione di funzione costo, algoritmo e valore di budget, è stato prima selezionato un seed set  $S$  tale che  $c(S) \leq k$ . Successivamente è stato simulato il processo di Majority Cascade sul grafo originale  $G$ . Le prestazioni sono state valutate considerando il numero finale di nodi attivati  $|Inf[G, S]|$  e il *diffusion ratio*, definito come:

$$\frac{|Inf[G, S]|}{|V|}.$$

In totale sono state eseguite:

$$3 \times 5 \times 6 = 90$$

configurazioni sperimentali, ottenute combinando tre funzioni costo, cinque algoritmi e sei livelli di budget. Tutti i seed set prodotti rispettano il vincolo di budget, quindi i risultati ottenuti sono ammissibili rispetto al problema considerato.

### 3.1 Confronto tra funzioni di costo

La prima parte dell'analisi confronta gli algoritmi separatamente per ciascuna funzione costo. Questa scelta permette di osservare quanto il modo in cui viene definito il costo dei nodi influenzi la selezione del seed set e, di conseguenza, la diffusione finale.

#### 3.1.1 Random Cost Function

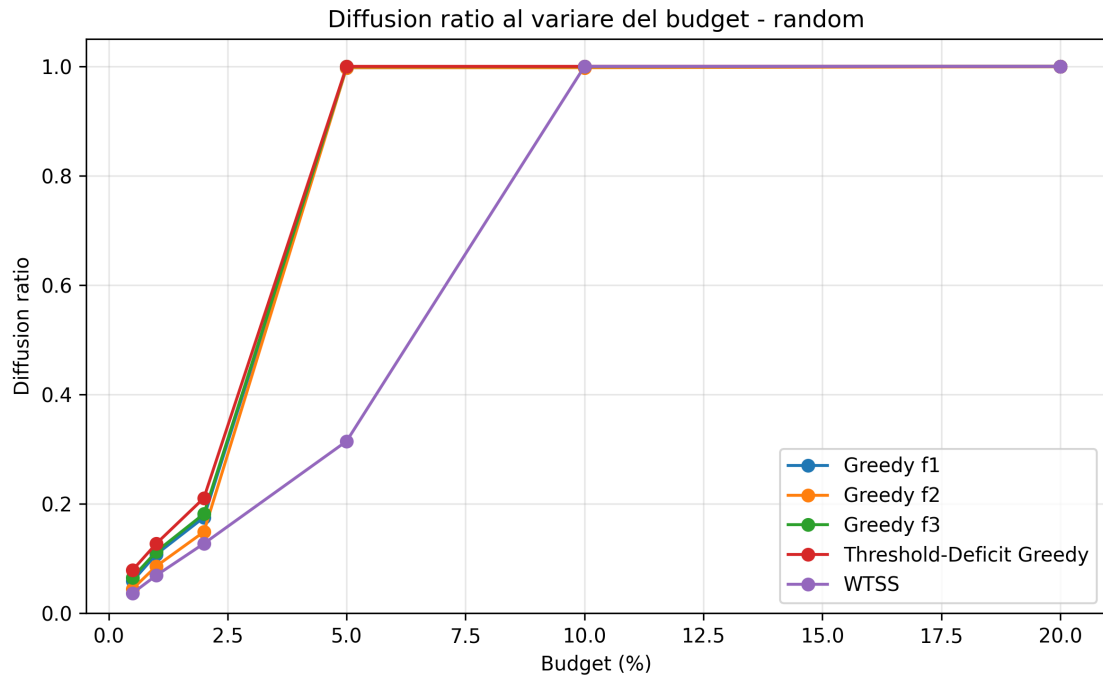
Nel caso della *Random Cost Function*, il costo dei nodi non dipende dalla struttura della rete. Ogni nodo riceve infatti un costo casuale compreso tra 1 e 10, senza considerare il grado o la posizione del nodo nel grafo. Questo scenario è utile per valutare cosa accade quando anche nodi potenzialmente molto influenti possono avere un costo basso.

I valori di *diffusion ratio* ottenuti con questa funzione costo sono riportati nella Tabella 4, mentre l'andamento complessivo è mostrato in Figura 3.



**Tabella 4:** Diffusion ratio al variare del budget con Random Cost Function.

| Budget $k$ | Greedy f1 | Greedy f2 | Greedy f3 | Threshold-Deficit Greedy | WTSS   |
|------------|-----------|-----------|-----------|--------------------------|--------|
| 300        | 0,0586    | 0,0443    | 0,0652    |                          | 0,0788 |
| 600        | 0,1078    | 0,0862    | 0,1127    |                          | 0,1273 |
| 1201       | 0,1752    | 0,1488    | 0,1817    |                          | 0,2106 |
| 3003       | 0,9976    | 0,9976    | 0,9995    |                          | 1,0000 |
| 6007       | 0,9985    | 0,9976    | 0,9995    |                          | 1,0000 |
| 12014      | 0,9995    | 0,9995    | 1,0000    |                          | 1,0000 |

**Figura 3:** Diffusion ratio al variare del budget con Random Cost Function.

Dalla tabella e dal grafico si nota che la diffusione cresce molto rapidamente. In particolare, *Threshold-Deficit Greedy* raggiunge l'attivazione completa della rete già con un budget pari al 5% del costo totale. Per  $k = 3003$ , infatti, l'algoritmo ottiene un *diffusion ratio* pari a 1, attivando tutti i 10.876 nodi della rete.

Questo risultato è legato alla natura della funzione costo randomica. Poiché il costo non dipende dalla posizione topologica dei nodi, può capitare che nodi importanti per la diffusione abbiano un costo ridotto. Di conseguenza, gli algoritmi possono costruire seed set molto efficaci anche con budget non particolarmente elevati.

**Risultato principale**

Con Random Cost Function, Threshold-Deficit Greedy è il migliore ai budget bassi e medi e raggiunge la diffusione completa già al 5%. Tuttavia, questo scenario è meno strutturale, perché il costo dei nodi non dipende dalla topologia della rete.

**3.1.2 Half-Node-Degree Cost Function**

Nel secondo scenario è stata utilizzata la funzione costo basata sul grado:

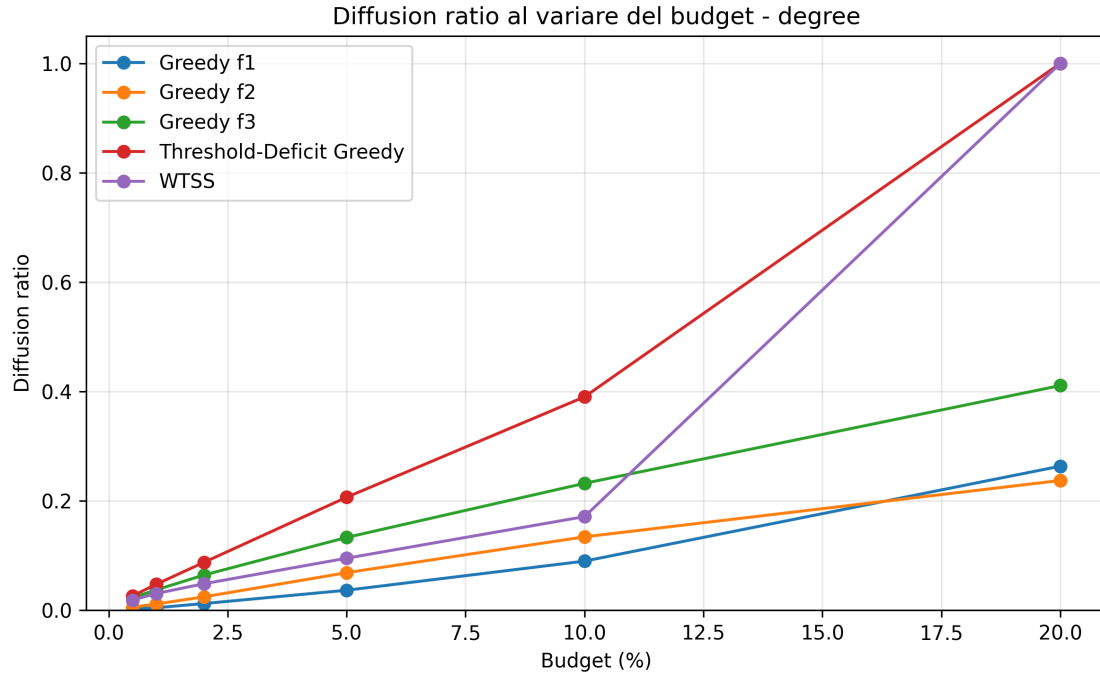
$$c(u) = \left\lceil \frac{d(u)}{2} \right\rceil.$$

Questa funzione, indicata anche come *Degree Cost*, assegna un costo più alto ai nodi con grado elevato. In questo modo, i nodi più connessi, che potrebbero essere utili per diffondere l'influenza, diventano anche più costosi da selezionare come seed.

I risultati sono riportati nella Tabella 5 e nella Figura 4.

**Tabella 5:** Diffusion ratio al variare del budget con Half-Node-Degree Cost Function.

| Budget $k$ | Greedy f1 | Greedy f2 | Greedy f3 | Threshold-Deficit Greedy | WTSS   |
|------------|-----------|-----------|-----------|--------------------------|--------|
| 215        | 0,0022    | 0,0054    | 0,0228    | 0,0263                   | 0,0187 |
| 431        | 0,0046    | 0,0115    | 0,0373    | 0,0475                   | 0,0306 |
| 863        | 0,0122    | 0,0245    | 0,0644    | 0,0875                   | 0,0485 |
| 2158       | 0,0366    | 0,0687    | 0,1331    | 0,2066                   | 0,0952 |
| 4316       | 0,0898    | 0,1342    | 0,2321    | 0,3902                   | 0,1712 |
| 8633       | 0,2634    | 0,2373    | 0,4109    | 1,0000                   | 1,0000 |



**Figura 4:** Diffusion ratio al variare del budget con Half-Node-Degree Cost Function.

Rispetto alla Random Cost, la crescita della diffusione è più lenta. Questo comportamento è atteso, perché la Degree Cost penalizza direttamente gli hub, rendendo più difficile selezionare nodi molto connessi senza esaurire rapidamente il budget.

Anche in questo caso, *Threshold-Deficit Greedy* ottiene i risultati migliori per i budget bassi e medi. Al 10% del budget, cioè per  $k = 4316$ , raggiunge un *diffusion ratio* pari a 0,3902, corrispondente a 4244 nodi attivati. Nella stessa configurazione, *Greedy f3* ottiene 0,2321, mentre WTSS arriva a 0,1712.

Al 20% del budget, sia WTSS sia *Threshold-Deficit Greedy* raggiungono la diffusione completa. In questo caso il solo *diffusion ratio* non basta più per distinguere le soluzioni, perché entrambi gli algoritmi raggiungono il valore massimo. Diventa quindi utile considerare anche il costo effettivamente usato e il tempo di esecuzione.

#### Risultato principale

Con Half-Node-Degree Cost Function, *Threshold-Deficit Greedy* è il migliore ai budget bassi e medi. Al 20%, WTSS e *Threshold-Deficit Greedy* raggiungono entrambi la diffusione completa, ma WTSS usa meno costo ed è più veloce.

### 3.1.3 Sublinear Degree Cost Function

La terza funzione considerata è la *Sublinear Degree Cost Function*, definita come:

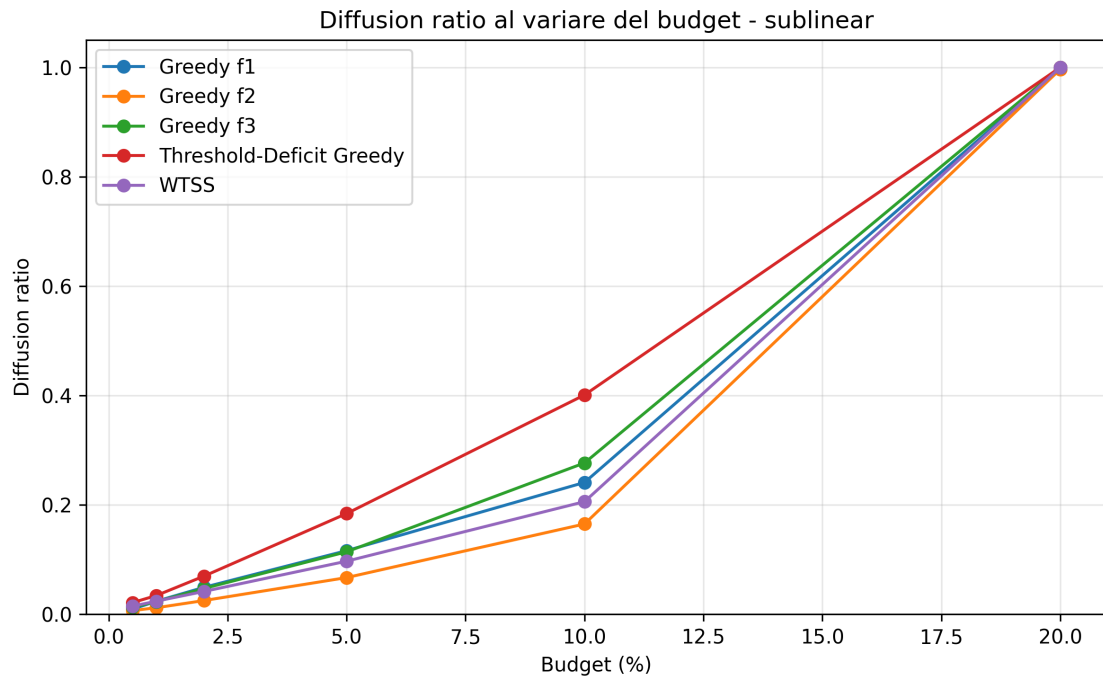
$$c_{sub}(u) = 1 + \lceil \log_2(1 + d(u)) \rceil.$$

Questa funzione è stata introdotta come compromesso tra Random Cost e Degree Cost. A differenza della Random Cost, mantiene un legame con la struttura della rete; rispetto alla Degree Cost, però, penalizza gli hub in modo meno forte.

I risultati sono riportati nella Tabella 6 e nella Figura 5.

**Tabella 6:** Diffusion ratio al variare del budget con Sublinear Degree Cost Function.

| Budget $k$ | Greedy f1 | Greedy f2 | Greedy f3 | Threshold-Deficit Greedy | WTSS   |
|------------|-----------|-----------|-----------|--------------------------|--------|
| 210        | 0,0095    | 0,0067    | 0,0128    | 0,0212                   | 0,0147 |
| 420        | 0,0237    | 0,0119    | 0,0230    | 0,0342                   | 0,0239 |
| 840        | 0,0492    | 0,0250    | 0,0471    | 0,0696                   | 0,0417 |
| 2100       | 0,1163    | 0,0670    | 0,1142    | 0,1841                   | 0,0970 |
| 4201       | 0,2410    | 0,1650    | 0,2767    | 0,4010                   | 0,2059 |
| 8402       | 0,9976    | 0,9967    | 0,9995    | 1,0000                   | 1,0000 |



**Figura 5:** Diffusion ratio al variare del budget con Sublinear Degree Cost Function.

I risultati mostrano che *Threshold-Deficit Greedy* è il migliore per tutti i budget fino

al 10%. In particolare, al 10% del budget, cioè per  $k = 4201$ , l'algoritmo ottiene un *diffusion ratio* pari a 0,4010, attivando 4361 nodi. Questo è il risultato più alto ottenuto al 10% tra le funzioni costo strutturali considerate.

Questo comportamento evidenzia l'utilità della Sublinear Degree Cost. Penalizzando gli hub in modo meno aggressivo rispetto alla Degree Cost, permette di selezionare nodi strutturalmente importanti senza consumare troppo rapidamente il budget. Allo stesso tempo, rimane più legata alla struttura della rete rispetto alla Random Cost.

Al 20%, sia WTSS sia Threshold-Deficit Greedy raggiungono la diffusione completa. Tuttavia, in questa configurazione Threshold-Deficit Greedy usa un costo pari a 4573, mentre WTSS usa un costo pari a 4939. Inoltre, in questa specifica configurazione Threshold-Deficit Greedy risulta anche più veloce.

#### Risultato principale

Con Sublinear Degree Cost Function, Threshold-Deficit Greedy ottiene i risultati più interessanti del progetto: è il migliore fino al 10% e raggiunge la diffusione completa al 20%, mantenendo un buon equilibrio tra diffusione, costo del seed set e tempo di esecuzione.

### 3.2 Confronto complessivo dei migliori risultati

Per riassumere il comportamento degli algoritmi, la Tabella 7 riporta, per ciascuna funzione costo e per ciascun valore di budget, l'algoritmo che ha ottenuto il miglior *diffusion ratio*. Nei casi in cui più algoritmi raggiungono lo stesso valore massimo, viene preferita la soluzione con costo del seed set inferiore.

**Tabella 7:** Migliori risultati per funzione costo e budget.

| Funzione costo | Budget | Algoritmo migliore       | Costo seed | Nodi attivati | Diffusion ratio |
|----------------|--------|--------------------------|------------|---------------|-----------------|
| Degree         | 0,5%   | Threshold-Deficit Greedy | 215        | 286           | 0,0263          |
| Degree         | 1%     | Threshold-Deficit Greedy | 431        | 517           | 0,0475          |
| Degree         | 2%     | Threshold-Deficit Greedy | 863        | 952           | 0,0875          |
| Degree         | 5%     | Threshold-Deficit Greedy | 2158       | 2247          | 0,2066          |
| Degree         | 10%    | Threshold-Deficit Greedy | 4316       | 4244          | 0,3902          |
| Degree         | 20%    | WTSS                     | 7044       | 10876         | 1,0000          |
| Random         | 0,5%   | Threshold-Deficit Greedy | 300        | 857           | 0,0788          |
| Random         | 1%     | Threshold-Deficit Greedy | 600        | 1385          | 0,1273          |
| Random         | 2%     | Threshold-Deficit Greedy | 1201       | 2291          | 0,2106          |
| Random         | 5%     | Threshold-Deficit Greedy | 2078       | 10876         | 1,0000          |
| Random         | 10%    | Threshold-Deficit Greedy | 2078       | 10876         | 1,0000          |
| Random         | 20%    | Threshold-Deficit Greedy | 2078       | 10876         | 1,0000          |
| Sublinear      | 0,5%   | Threshold-Deficit Greedy | 210        | 231           | 0,0212          |
| Sublinear      | 1%     | Threshold-Deficit Greedy | 420        | 372           | 0,0342          |
| Sublinear      | 2%     | Threshold-Deficit Greedy | 840        | 757           | 0,0696          |
| Sublinear      | 5%     | Threshold-Deficit Greedy | 2100       | 2002          | 0,1841          |
| Sublinear      | 10%    | Threshold-Deficit Greedy | 4200       | 4361          | 0,4010          |
| Sublinear      | 20%    | Threshold-Deficit Greedy | 4573       | 10876         | 1,0000          |

Dalla tabella emerge che Threshold-Deficit Greedy è l'algoritmo più efficace nella maggior parte degli scenari, soprattutto per budget bassi e medi. Questo risultato è coerente con l'idea dell'euristica, che sceglie i nodi considerando il deficit di attivazione dei vicini rispetto alla soglia majority.

Quando il budget è elevato, più algoritmi riescono a raggiungere la diffusione completa. In questi casi il *diffusion ratio* da solo non è sufficiente per stabilire quale soluzione sia migliore, perché il valore ottenuto è lo stesso. Per questo motivo è utile considerare anche il costo effettivamente utilizzato e il tempo di esecuzione.

### 3.3 Analisi dei tempi di esecuzione

Oltre alla qualità della diffusione, è stato considerato anche il tempo di esecuzione degli algoritmi. La Tabella 8 riporta il runtime medio osservato per ciascun algoritmo.

**Tabella 8:** Tempo medio di esecuzione per algoritmo.

| Algoritmo                | Runtime medio (s) |
|--------------------------|-------------------|
| Greedy f2                | 61,43             |
| Greedy f3                | 55,93             |
| Greedy f1                | 45,43             |
| Threshold-Deficit Greedy | 12,23             |
| WTSS                     | 9,46              |

Dai risultati si nota che le tre varianti greedy sono le più costose dal punto di vista computazionale. Questo dipende dal fatto che, a ogni iterazione, esse devono valutare il guadagno marginale dei nodi candidati.

WTSS è mediamente l'algoritmo più veloce. Tuttavia, non sempre ottiene i risultati migliori in termini di diffusione, soprattutto con budget bassi e medi. Threshold-Deficit Greedy è leggermente più lento di WTSS in media, ma spesso produce seed set più efficaci. Per questo motivo rappresenta un buon compromesso tra qualità della diffusione e tempo di esecuzione.

#### Risultato principale

WTSS è l'algoritmo più veloce in media, mentre Threshold-Deficit Greedy offre il miglior compromesso tra diffusione ottenuta e runtime. Le varianti Greedy f1, f2 e f3 risultano invece più costose computazionalmente.

### 3.4 Valutazione della robustezza

La seconda parte dell'analisi riguarda la robustezza dei seed set selezionati. In particolare, sono state considerate due configurazioni rappresentative:

- **Degree Cost + Greedy f3 + budget 10%;**
- **Sublinear Degree Cost + Threshold-Deficit Greedy + budget 10%.**

La prima configurazione rappresenta il migliore risultato ottenuto tra le varianti greedy standard in uno scenario strutturale basato sul grado. La seconda rappresenta invece la combinazione più interessante proposta nel progetto, perché unisce la funzione costo sublineare con l'euristica Threshold-Deficit Greedy.

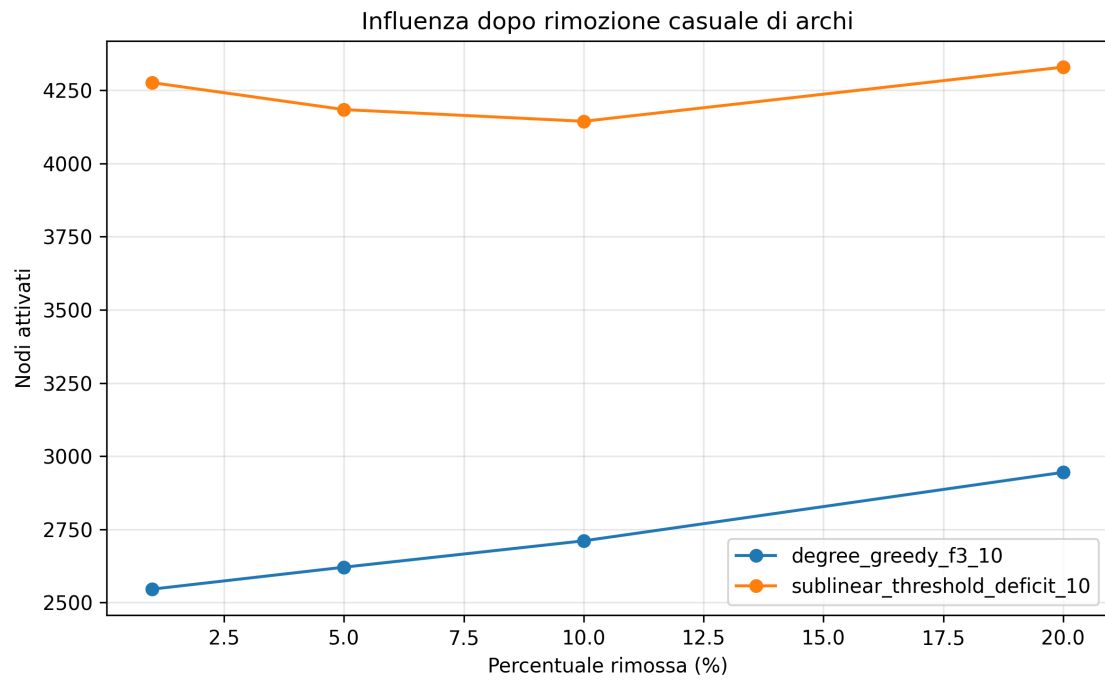
### 3.4.1 Rimozione casuale di archi

Nel primo stress test è stata effettuata la rimozione casuale dell'1%, 5%, 10% e 20% degli archi. Il seed set selezionato sul grafo originale  $G$  è stato mantenuto invariato e il processo di Majority Cascade è stato rieseguito sul grafo perturbato  $G'$ .

I risultati sono riportati nella Tabella 9 e rappresentati graficamente nelle Figure 6 e 7.

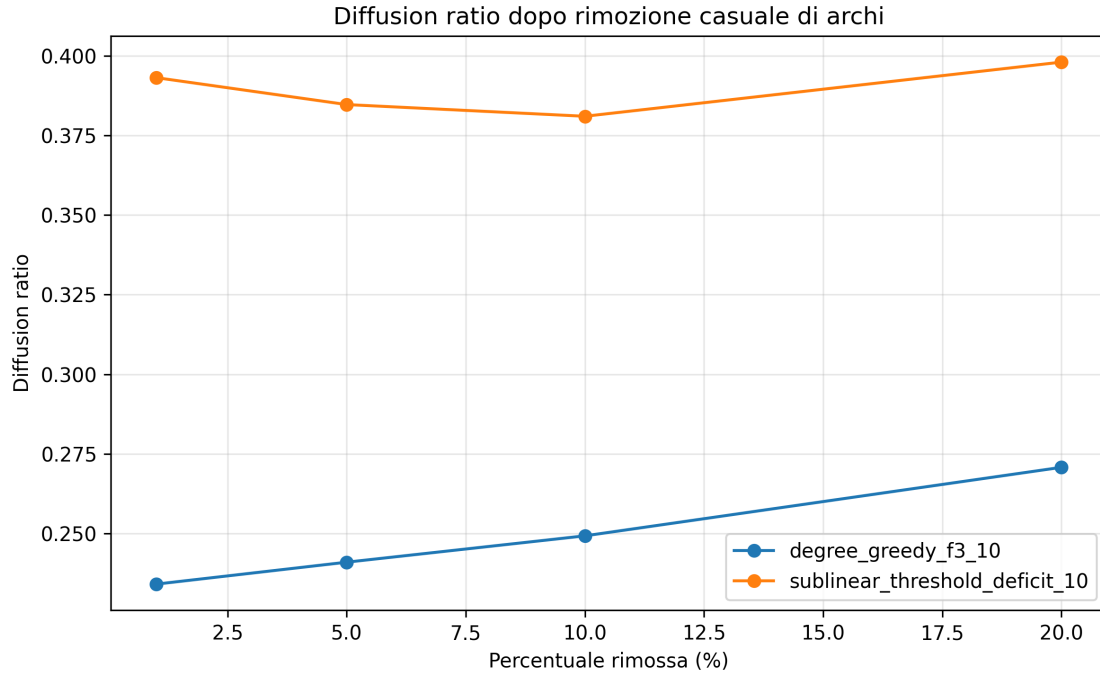
**Tabella 9:** Risultati dello stress test con rimozione casuale di archi.

| Configurazione                | Archi rimossi (%) | Inf. originale | Inf. modificata | Ratio originale | Ratio modificato | Variazione |
|-------------------------------|-------------------|----------------|-----------------|-----------------|------------------|------------|
| Degree + Greedy f3            | 1%                | 2524           | 2546            | 0,2321          | 0,2341           | +22        |
| Degree + Greedy f3            | 5%                | 2524           | 2621            | 0,2321          | 0,2410           | +97        |
| Degree + Greedy f3            | 10%               | 2524           | 2711            | 0,2321          | 0,2493           | +187       |
| Degree + Greedy f3            | 20%               | 2524           | 2945            | 0,2321          | 0,2708           | +421       |
| Sublinear + Threshold-Deficit | 1%                | 4361           | 4276            | 0,4010          | 0,3932           | -85        |
| Sublinear + Threshold-Deficit | 5%                | 4361           | 4184            | 0,4010          | 0,3847           | -177       |
| Sublinear + Threshold-Deficit | 10%               | 4361           | 4144            | 0,4010          | 0,3810           | -217       |
| Sublinear + Threshold-Deficit | 20%               | 4361           | 4329            | 0,4010          | 0,3980           | -32        |



**Figura 6:** Numero di nodi attivati dopo rimozione casuale di archi.





**Figura 7:** Diffusion ratio dopo rimozione casuale di archi.

Nel caso della configurazione Degree Cost + Greedy f3, la rimozione di archi porta a un aumento progressivo dell'influenza. Si passa infatti da 2524 nodi attivati sul grafo originale a 2945 nodi attivati dopo la rimozione del 20% degli archi.

Questo risultato può sembrare controintuitivo, ma è coerente con il modello Majority Cascade. Rimuovendo archi, il grado di alcuni nodi diminuisce e, di conseguenza, può diminuire anche la loro soglia di attivazione:

$$\tau(v) = \left\lceil \frac{d(v)}{2} \right\rceil.$$

Alcuni nodi possono quindi diventare più facili da attivare rispetto al grafo originale.

Nel caso della configurazione Sublinear Degree Cost + Threshold-Deficit Greedy, la diffusione diminuisce leggermente per le prime percentuali di rimozione, ma rimane vicina al valore iniziale. Anche dopo la rimozione del 20% degli archi, vengono attivati 4329 nodi rispetto ai 4361 del grafo originale. Questo indica che il seed set scelto da Threshold-Deficit Greedy rimane abbastanza stabile rispetto alla rimozione casuale di archi.

### 3.4.2 Rimozione casuale di nodi

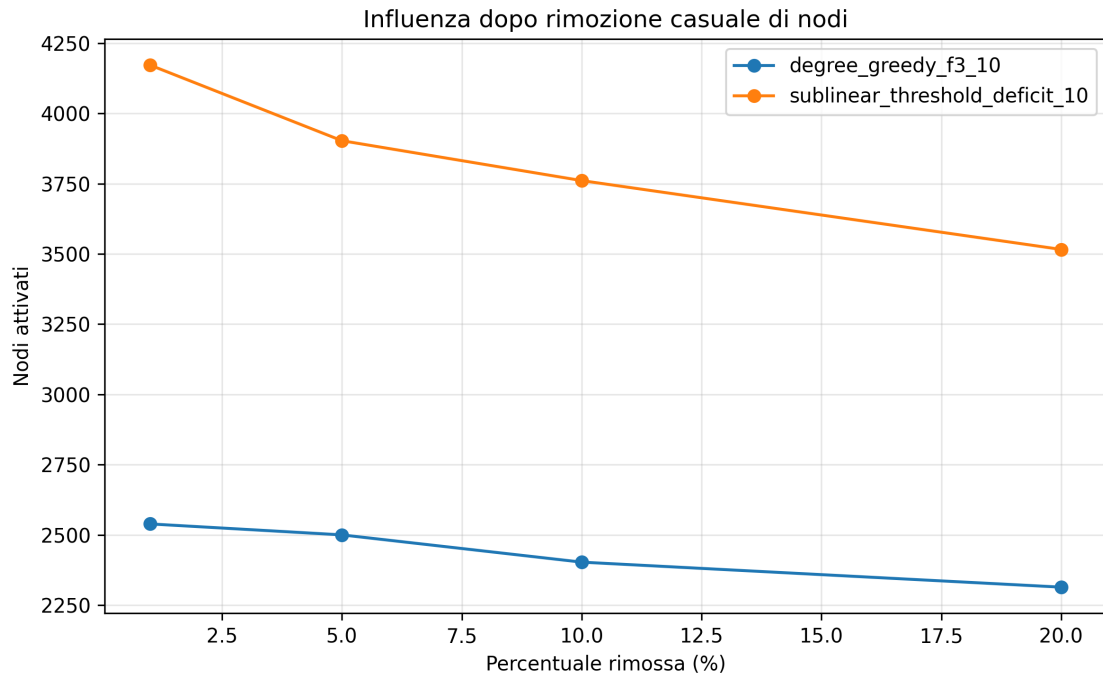
Nel secondo stress test è stata effettuata la rimozione casuale dell'1%, 5%, 10% e 20% dei nodi. In questo caso, poiché alcuni nodi del seed set possono essere rimossi dal grafo, il seed set utilizzato sul grafo perturbato è:

$$S' = S \cap V(G').$$

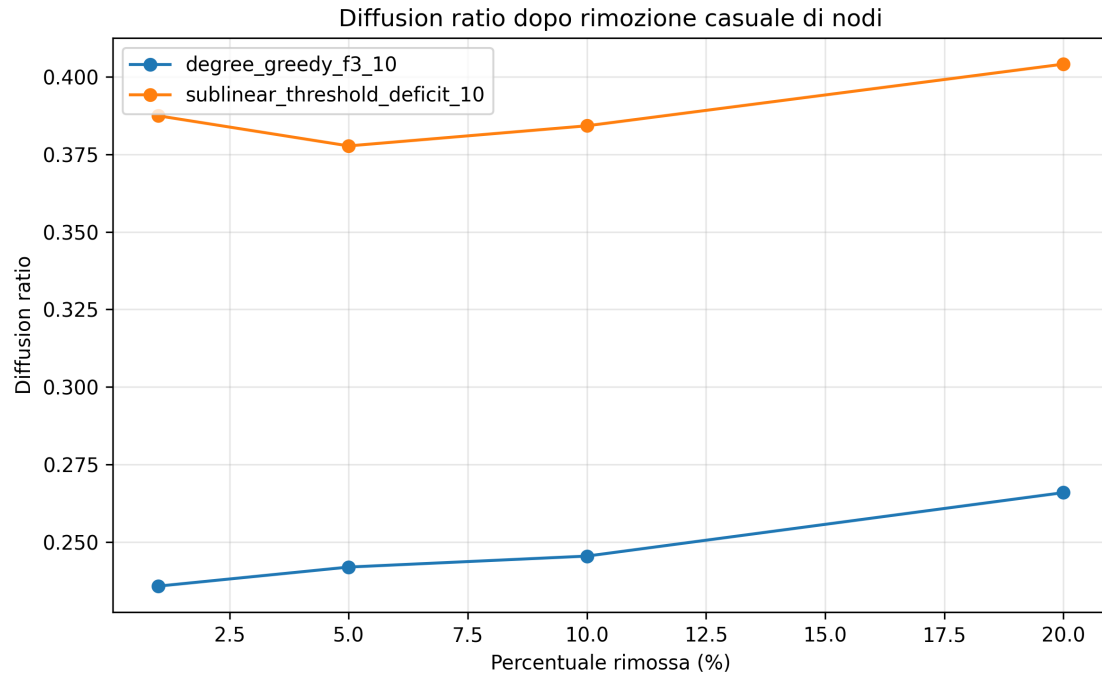
I risultati sono riportati nella Tabella 10 e nelle Figure 8, 9 e 10.

**Tabella 10:** Risultati dello stress test con rimozione casuale di nodi.

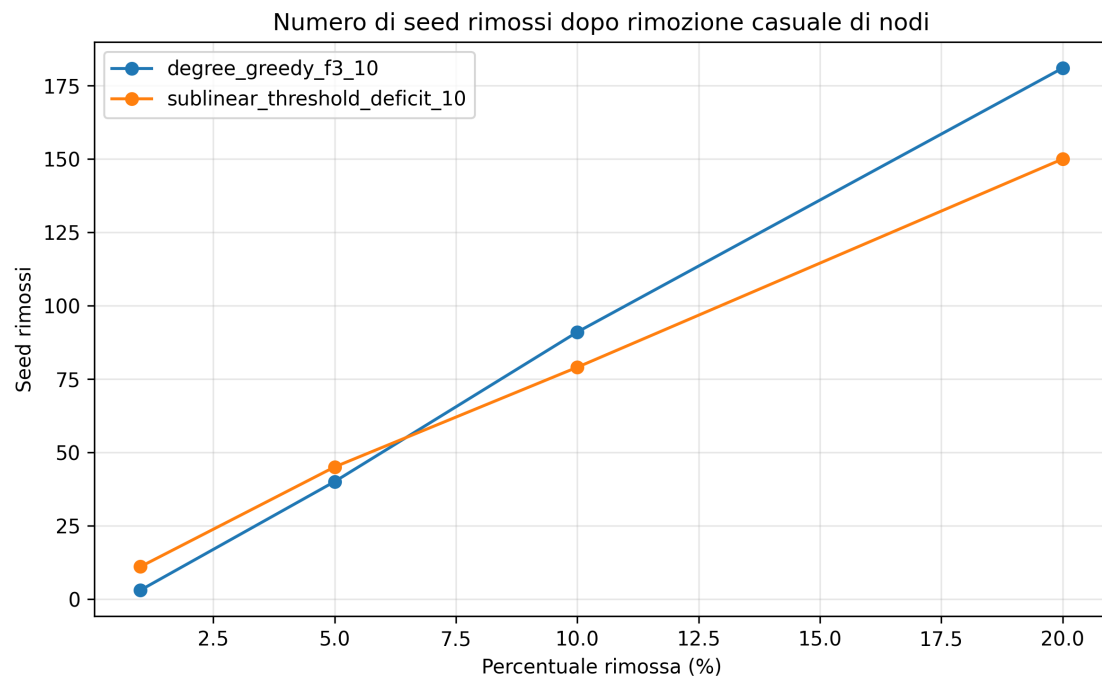
| Configurazione                | Nodi rimossi (%) | Seed rimossi | Inf. originale | Inf. modificata | Ratio originale | Ratio modificato | Variazione |
|-------------------------------|------------------|--------------|----------------|-----------------|-----------------|------------------|------------|
| Degree + Greedy f3            | 1%               | 3            | 2524           | 2539            | 0,2321          | 0,2358           | +15        |
| Degree + Greedy f3            | 5%               | 40           | 2524           | 2500            | 0,2321          | 0,2419           | -24        |
| Degree + Greedy f3            | 10%              | 91           | 2524           | 2403            | 0,2321          | 0,2455           | -121       |
| Degree + Greedy f3            | 20%              | 181          | 2524           | 2314            | 0,2321          | 0,2659           | -210       |
| Sublinear + Threshold-Deficit | 1%               | 11           | 4361           | 4172            | 0,4010          | 0,3874           | -189       |
| Sublinear + Threshold-Deficit | 5%               | 45           | 4361           | 3903            | 0,4010          | 0,3777           | -458       |
| Sublinear + Threshold-Deficit | 10%              | 79           | 4361           | 3761            | 0,4010          | 0,3842           | -600       |
| Sublinear + Threshold-Deficit | 20%              | 150          | 4361           | 3516            | 0,4010          | 0,4041           | -845       |



**Figura 8:** Numero di nodi attivati dopo rimozione casuale di nodi.



**Figura 9:** Diffusion ratio dopo rimozione casuale di nodi.



**Figura 10:** Numero di seed rimossi durante lo stress test sui nodi.

Nel caso della rimozione di nodi, il numero assoluto di nodi attivati tende in generale a diminuire. Questo è prevedibile, perché la rimozione può eliminare sia nodi appartenenti al seed set sia nodi utili alla propagazione della cascata.

Tuttavia, il *diffusion ratio* può aumentare anche quando il numero assoluto di nodi attivati diminuisce. Questo accade perché il rapporto viene calcolato rispetto al numero di nodi rimasti nel grafo modificato. Ad esempio, nella configurazione Degree Cost + Greedy f3, con la rimozione del 20% dei nodi l'influenza assoluta passa da 2524 a 2314 nodi, ma il *diffusion ratio* cresce da 0,2321 a 0,2659.

Per la configurazione Sublinear Degree Cost + Threshold-Deficit Greedy, la perdita assoluta è più alta, ma il numero di nodi attivati rimane comunque superiore rispetto alla configurazione Degree Cost + Greedy f3. Anche dopo la rimozione del 20% dei nodi, vengono attivati 3516 nodi.

### 3.5 Discussione finale

L'analisi dei risultati mette in evidenza tre aspetti principali.

Il primo riguarda il ruolo della funzione costo. La Random Cost permette di raggiungere rapidamente la saturazione della rete, ma rappresenta uno scenario meno legato alla struttura del grafo. La Degree Cost rende invece il problema più difficile, perché penalizza fortemente i nodi ad alto grado. La Sublinear Degree Cost risulta il compromesso più interessante, perché mantiene un legame con la struttura della rete senza rendere troppo costosi gli hub.

Il secondo aspetto riguarda gli algoritmi. Threshold-Deficit Greedy è l'algoritmo più competitivo nella maggior parte degli scenari, soprattutto per budget bassi e medi. Questo risultato dipende dal fatto che l'euristica usa informazioni direttamente legate al Majority Cascade, considerando quanto i vicini dei nodi candidati siano vicini alla propria soglia di attivazione.

Il terzo aspetto riguarda gli stress test. Le modifiche strutturali della rete possono produrre effetti non sempre immediati da prevedere. In particolare, la rimozione di archi può talvolta aumentare la diffusione, perché riduce i gradi dei nodi e può quindi abbassare le soglie majority. La rimozione di nodi tende invece a ridurre il numero assoluto di attivati, ma il *diffusion ratio* può comunque aumentare perché il grafo modificato contiene meno nodi.

**Risultato principale**

Il risultato complessivo più rilevante è che la combinazione Sublinear Degree Cost + Threshold-Deficit Greedy rappresenta il miglior compromesso tra coerenza della funzione costo, qualità della diffusione, rispetto del budget e costo computazionale. Questa configurazione sfrutta sia la struttura della rete sia la logica del Majority Cascade, ottenendo risultati molto competitivi soprattutto ai budget bassi e medi.

## 4 Conclusioni

In questo lavoro è stato affrontato il problema della massimizzazione dell'influenza in reti complesse tramite il modello Majority Cascade con vincolo di costo. L'obiettivo era selezionare un *seed set*  $S \subseteq V$  capace di attivare il maggior numero possibile di nodi, rispettando un budget prefissato  $c(S) \leq k$ . Gli esperimenti sono stati svolti sulla rete reale **p2p-Gnutella04**, trattata come grafo non orientato perché il modello utilizzato si basa su vicinato, grado e soglia di maggioranza.

Sono state considerate tre funzioni costo: *Random Cost*, *Degree Cost* e *Sublinear Degree Cost*. La *Random Cost* assegna costi indipendenti dalla struttura della rete; la *Degree Cost* penalizza direttamente i nodi ad alto grado; la *Sublinear Degree Cost*, invece, mantiene un legame con la topologia del grafo ma penalizza gli hub in modo meno forte. Dal punto di vista algoritmico sono stati confrontati cinque metodi: le tre varianti *Greedy f1*, *Greedy f2* e *Greedy f3*, l'algoritmo *WTSS* e l'euristica proposta *Threshold-Deficit Greedy*.

Dai risultati emerge che la scelta della funzione costo ha un impatto importante sulla diffusione finale. Con la *Random Cost*, la rete raggiunge la saturazione molto rapidamente. In particolare, *Threshold-Deficit Greedy* ottiene i risultati migliori ai budget bassi e medi e riesce ad attivare tutta la rete già con un budget pari al 5% del costo totale. Questo risultato è dovuto al fatto che il costo dei nodi non dipende dalla loro posizione nella rete: nodi molto influenti possono quindi avere costo basso ed essere selezionati facilmente. Tuttavia, questa funzione costo è meno significativa dal punto di vista strutturale.

Con la *Degree Cost*, il problema diventa più difficile, perché i nodi ad alto grado, potenzialmente utili alla diffusione, diventano anche più costosi. In questo scenario, *Threshold-Deficit Greedy* risulta il migliore per budget bassi e medi. Al 10% del budget raggiunge un *diffusion ratio* pari a 0,3902, superiore a quello ottenuto dalle varianti greedy e da WTSS. Al 20%, invece, sia WTSS sia *Threshold-Deficit Greedy* raggiungono l'attivazione completa della rete; in questo caso, oltre al *diffusion ratio*, diventa importante considerare anche il costo effettivamente usato e il tempo di esecuzione.

La *Sublinear Degree Cost* rappresenta il caso più interessante. Questa funzione mantiene una relazione con la struttura della rete, ma evita di penalizzare eccessivamente gli hub. Con questa funzione costo, *Threshold-Deficit Greedy* ottiene i risultati migliori fino al 10% del budget, dove attiva 4361 nodi e raggiunge un *diffusion ratio* pari a 0,4010. Al 20% raggiunge invece la diffusione completa usando solo una parte

del budget disponibile. Questo conferma che la combinazione tra Sublinear Degree Cost e Threshold-Deficit Greedy è la più significativa tra quelle analizzate.

Un altro aspetto emerso riguarda i tempi di esecuzione. Le varianti greedy classiche risultano più costose dal punto di vista computazionale, perché richiedono di valutare più volte il guadagno marginale dei nodi candidati. WTSS è mediamente l'algoritmo più veloce, ma non sempre ottiene i migliori risultati in termini di diffusione. Threshold-Deficit Greedy, pur essendo leggermente più lento di WTSS, offre un buon compromesso tra qualità della diffusione e tempo di esecuzione, soprattutto nei budget bassi e medi.

Gli stress test hanno mostrato che il comportamento del Majority Cascade può non essere sempre intuitivo. Nel caso della rimozione casuale di archi, la diffusione non diminuisce necessariamente: in alcuni casi può anche aumentare, perché la rimozione di collegamenti riduce il grado dei nodi e quindi può abbassare la soglia majority  $\lceil d(v)/2 \rceil$ . Nel caso della rimozione casuale di nodi, invece, il numero assoluto di nodi attivati tende generalmente a diminuire, anche perché alcuni seed possono essere rimossi dal grafo. Tuttavia, il *diffusion ratio* può aumentare, poiché viene calcolato rispetto a un grafo modificato con meno nodi.

In conclusione, i risultati mostrano che la funzione costo e l'algoritmo di selezione del seed set incidono in modo determinante sulla massimizzazione dell'influenza. La Random Cost permette una diffusione rapida, ma è meno legata alla struttura della rete. La Degree Cost rende il problema più selettivo, penalizzando fortemente gli hub. La Sublinear Degree Cost risulta invece il compromesso più equilibrato, perché considera la struttura del grafo senza rendere troppo costosi i nodi centrali.

La conclusione principale del progetto è che la combinazione **Sublinear Degree Cost + Threshold-Deficit Greedy** rappresenta il miglior compromesso tra coerenza strutturale, qualità della diffusione, rispetto del budget e costo computazionale. Questa configurazione sfrutta sia la topologia della rete sia la logica del Majority Cascade, ottenendo risultati competitivi soprattutto in scenari con budget limitato o intermedio.

Come possibile sviluppo futuro, sarebbe interessante ripetere gli esperimenti su altri dataset reali, in modo da verificare se gli stessi comportamenti si osservano anche su reti con caratteristiche topologiche diverse. Inoltre, gli stress test potrebbero essere ripetuti su più istanze casuali, così da ottenere una valutazione ancora più stabile della robustezza dei seed set selezionati.